Lecture 1: Time Complexity & C++ STL Basics

Basics of CP Track

1 Counting Operations & Time Complexity

1.1 Brute Force Example: Two Sum

Given an array of N integers, find if there are two numbers that add up to a target sum X.

```
int count = 0;
for (int i = 0; i < n; i++) {
    for (int j = i + 1; j < n; j++) {
        if (arr[i] + arr[j] == X) {
            count++;
        }
    }
}
```

How many operations does this take?

- When $i = 0$, inner loop runs $N - 1$ times.
- When $i = 1$, inner loop runs $N - 2$ times.
- Total $= (N-1) + (N-2) + \cdots + 1 = \frac{N(N-1)}{2} \approx O(N^2)$.

1.2 The CP Rule of Thumb

A modern judge can perform roughly 10^8 operations per second.

If $N = 10^5$, an $O(N^2)$ solution takes $\approx 10^{10}$ operations, requiring ~ 100 seconds. This will result in a **Time Limit Exceeded (TLE)**. We need algorithms closer to $O(N \log N)$ or $O(N)$. *We will learn techniques ahead that help solve this problem much faster!*

1.3 Linear vs. Binary Search

Searching for a number in a **sorted** array of size $N = 10^6$:

- **Linear Search:** Checks every element. Worst case: 10^6 operations ($O(N)$).
- **Binary Search:** Halves the search space every step. Worst case: $\log_2(10^6) \approx 20$ operations ($O(\log N)$).

2 The C++ STL (Standard Template Library)

2.1 `vector` and Iterators

A dynamic array.

- `push_back(x)`: Adds to the end ($O(1)$).
- `v[i]`: Access element at index i ($O(1)$).
- `pop_back()`: Removes the last element ($O(1)$).
- `insert()`: Inserts at a position ($O(N)$ - *Avoid inside loops!*).

Iterators: Pointers to elements in the container.

- `v.begin()`: Points to the first element.
- `v.end()`: Points to *one past* the last element.

Sorting & Binary Search (Requires sorted array):

```
sort(v.begin(), v.end()); // O(N log N)
auto it1 = lower_bound(v.begin(), v.end(), val); // First element >= val
auto it2 = upper_bound(v.begin(), v.end(), val); // First element > val
```

2.2 `stack`

LIFO (Last-In, First-Out) structure.

- `push(x)`: Add to top ($O(1)$).
- `pop()`: Remove from top ($O(1)$).
- `top()`: Look at top element ($O(1)$).

Application Idea: Finding the **Nearest Smaller Values** (Monotonic Stack) - a classic CSES problem.

2.3 `queue`

FIFO (First-In, First-Out) structure.

- `push(x)`: Add to back ($O(1)$).
- `pop()`: Remove from front ($O(1)$).
- `front()`: Look at front element ($O(1)$).

Note: We won't do practice problems for this today. We will revisit queues extensively when we reach Breadth-First Search (BFS) in the Graphs section.

2.4 Pairs & Custom Comparators

Useful for binding two pieces of data, like coordinates or an element and its original index.

```
vector<pair<int, int>> v;
v.push_back({5, 2});
// By default, sorts by first element, then second.
```

Custom Comparator Example (Sorting pairs by the second element):

```
bool comp(const pair<int, int>& a, const pair<int, int>& b) {
    return a.second < b.second;
}
sort(v.begin(), v.end(), comp);
```

Crucial Rule: Your comparator **must** return `false` if the elements are equal (i.e., `comp(a, a) == false`). Using `<=` will cause a Segmentation Fault!

2.5 Set, Multiset, and Map

Under the hood, these are balanced Binary Search Trees. Operations are $O(\log N)$.

- `set`: Maintains a sorted collection of *unique* elements.
- `multiset`: Maintains a sorted collection, allows *duplicates*.
- `map`: Key-value pairs sorted by key. Useful for frequency counting (`freq[x]++`).

---

3 C++ Syntax Survival Guide

These are the "obvious" syntax tricks that competitive programmers use everywhere, but can look like magic to beginners.

3.1 Range-Based For Loops

Instead of writing `for(int i = 0; i < v.size(); i++)`, C++11 introduced a much cleaner way to iterate through collections:

```
vector<int> v = {1, 2, 3, 4, 5};

// 1. By Value (Makes a copy, safe but slower for big data)
for (int x : v) {
    cout << x << " ";
}

// 2. By Reference (Avoids copies, allows modifying original data)
for (auto& x : v) {
    x *= 2; // This permanently changes elements in 'v'
}
```

3.2 Iterators: Referencing and Incrementing

An iterator is essentially a pointer to an element inside a container. We use the `auto` keyword to avoid typing massive types like `vector<int>::iterator`.

```
auto it = v.begin(); // Points to the first element

// Dereferencing: Use the '*' operator to get the actual value
cout << *it << "\n";

// Incrementing: Move the iterator to the next element
it++;
```

Crucial Trap for Sets and Maps: For a `vector`, you can jump around by doing `it += 5` (random access). For a `set` or `map`, elements are stored in a tree, not an array. **You cannot do** `it + 5`. You must step through them using `it++` or `++it`.

3.3 Iterators and Pairs (-> Operator)

When an iterator points to a `pair` (which is what a `map` stores), dereferencing looks slightly different. You use the arrow operator (`->`):

```
map<string, int> m;
m["Alice"] = 10;

auto it = m.begin(); // Iterator points to a pair<string, int>

// These two lines do the exact same thing:
cout << (*it).first << " " << (*it).second << "\n";
cout << it->first << " " << it->second << "\n"; // Much cleaner!
```

---

4 Homework

Target the **CSES Problem Set**:

1. **Introductory Problems:** Complete the entire section to solidify basic implementation.
2. **Sorting and Searching (Selected based on today's lecture):**
   - *Distinct Numbers* (Apply `set` or `sort`)
   - *Sum of Two Values* (Apply `map` or sorting + `lower_bound`)
   - *Restaurant Customers* (Apply `vector<pair<int, int>>` and custom sorting)
   - *Nearest Smaller Values* (Apply `stack`)